

Immutable Images

An image is the container world's version of a commit — build it once, tag it, and that tag means the same bytes forever. Two compose files make that true in practice.

concept doc — no execution in this step 2026-08-16

01 build: describes how; image: names what

`build:` tells Compose how to produce something from source. `image:` names a specific, already-made thing. A service can carry both — Compose uses `build:` when you say `build`, and `image:` when you say `push` or `pull`. That's the entire trick: one file, two modes, no duplication.

02 Dev compose vs. deploy compose

`docker-compose.yml` — dev

```
coder-commands-mcp:
  build: ./coder-commands-mcp
  image: ...:${IMAGE_TAG:-dev}

has source — can build locally
```

`docker-compose.deploy.yml`

```
coder-commands-mcp:
  image: ...:${IMAGE_TAG}

no build: key at all —
nothing to build from, or with
```

This is the file that actually runs on clubuntu (\$06 of the main plan). It resolves every service to an already-built image; there's no source tree for it to need.

03 Why not :latest

Tagging every build by commit SHA instead of overwriting one `:latest` tag means the registry accumulates a short history of exactly-reproducible builds — which is what makes

rollback in the next two docs a pull instead of a rebuild.

:a1b2c3d
older build

:c9f0e21
older build

:f77ab30
deployed now

all still pullabl
any one is a roll

A single moving `:latest` tag can't do this — pulling it later gets whatever's newest, not what you tested.

04 The two files, concretely

DOCKER-COMPOSE.YML — ONE REPRESENTATIVE SERVICE

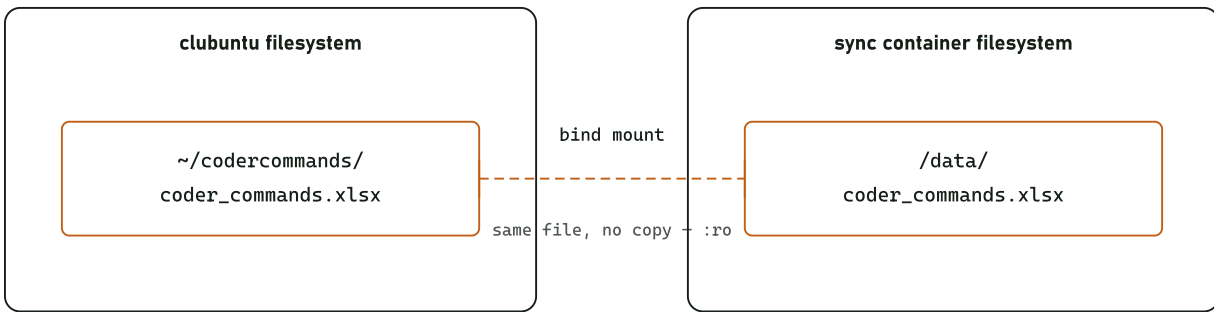
```
coder-commands-mcp:
  build: ./coder-commands-mcp
  image: clubuntu.dala-cirius.ts.net:5000/coder-commands-mcp:${IMAGE_TAG:-dev}
  container_name: coder-commands-mcp
  ...
```

DOCKER-COMPOSE.DEPLOY.YML — NEW, ABRIDGED

```
name: codercommands
services:
  postgres:
    image: postgres:17
    ...
  coder-commands-mcp:
    image: clubuntu.dala-cirius.ts.net:5000/coder-commands-mcp:${IMAGE_TAG}
    ...
# gateway, docker-mcp, postgres-mcp, sync follow the same shape
```

05 Data isn't part of the image

Three things on clubuntu are deliberately outside everything above: `.env`, the Postgres data volume, and `coder_commands.xlsx` — the spreadsheet `sync` ingests. None of them are baked into an image or travel through git; they're persistent state that lives directly on the target machine, updated in place, never rebuilt.



Same mechanism already in `docker-compose.yml` today, just a different file — `docker-mcp`'s `/var/run/docker.sock` bind mount works identically. An edit to the host file is instantly visible to the container's next read, which is what lets `sync`'s mtime-polling loop work at all.

Updating the spreadsheet is a direct copy to clubuntu, outside this entire pipeline — `scp coder_commands.xlsx clubuntu.dala-cirius.ts.net:~/codercommands/coder_commands.xlsx`. `sync` picks it up on its next poll; nothing here needs to build, tag, or deploy anything for a spreadsheet edit to take effect.